

A Resilience-Oriented Orchestration Framework for Mission Continuity in Multi-agent Autonomous Systems

Anjali Santhosh[✉]

University of L'Aquila

Dipartimento di Ingegneria e Scienze
dell'Informazione e Matematica
L'Aquila, Italy

anjali.santhosh@graduate.univaq.it

Massimo Tivoli

University of L'Aquila

Dipartimento di Ingegneria e Scienze
dell'Informazione e Matematica
L'Aquila, Italy

massimo.tivoli@univaq.it

Marco Autili

University of L'Aquila

Dipartimento di Ingegneria e Scienze
dell'Informazione e Matematica
L'Aquila, Italy

marco.autili@univaq.it

Abstract

Autonomous Multi-Agent Systems (MAS) operating in dynamic environments require resilient coordination mechanisms to maintain mission continuity despite operational disruptions. This paper presents a resilient orchestration framework that integrates runtime monitoring, interruption detection, recovery re-assessment, and re-orchestration within a unified lifecycle coordinated by a stateless Asynchronous Event Bus (AEB). By treating resilience as an intrinsic orchestration process, the framework enables recovery decision re-assessment and autonomous standby promotion without centralized replanning activities. Experimental evaluation in heterogeneous Search And Rescue (SAR) simulations demonstrates preservation of effective mission continuity and resilient recovery under varying failure conditions.

CCS Concepts

• **Computer systems organization** → **Reliability**; **Robotic autonomy**.

Keywords

Autonomous Multi-agent Systems, Mission Continuity Lifecycle, Task Re-orchestration, Resilient Orchestration

ACM Reference Format:

Anjali Santhosh, Massimo Tivoli, and Marco Autili. 2026. A Resilience-Oriented Orchestration Framework for Mission Continuity in Multi-agent Autonomous Systems. In *Proceedings of the 1st International Workshop on Resilience for Safe, Secure and Responsible Systems (RSSR '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3843780.3844537>

1 Introduction

Autonomous Multi-Agent Systems (MAS) have emerged as a fundamental paradigm for developing intelligent systems that perform complex missions by coordinating multiple autonomous agents. This results in a growing deployment of MAS in critical mission domains such as Search And Rescue (SAR), disaster response, environmental monitoring, emergency logistics, intelligent transportation, and autonomous robotic operations, where multiple heterogeneous agents must cooperate and coordinate to achieve mission

objectives under dynamic and uncertain conditions [14]. With the increasing autonomous deployment of MAS, the main challenge has changed from agent coordination and efficient task allocation [3] to maintaining mission continuity despite failures, resource degradation, environmental uncertainty, and operational disruptions. This can ultimately cause the entire mission to fail if the systems fail to detect these interruptions and prevent their propagation to subsequent stages of the mission [11]. Hence, autonomous MAS deployed in a dynamic environment must provide sufficient resilience mechanisms capable of continuous monitoring, detection, and recovery [9].

Recent advances in MAS have significantly improved distributed task allocation, coordination strategies, resource management, and autonomous decision-making. Despite these advances, resilience and recovery have often been addressed as secondary concerns layered onto coordination and allocation mechanisms, rather than as first-class design objectives, in some existing approaches [5, 15, 17]. Runtime monitoring, failure detection, recovery planning, re-allocation, and validation activities are often implemented independently, fragmenting resilience across mission execution. These include monitoring flag degradation without informing recovery decisions, recovery restoring failed components without considering system-level mission continuity [7], and recovery actions being executed without systematically validating their safety or efficiency under current mission conditions [8].

In this paper, we investigate mission continuity as the fundamental resilience objective of autonomous MAS to address these challenges. We approach (i) mission continuity as the ability of an autonomous system to sustain progress toward mission objectives despite operational disruptions through timely detection, recovery, and adaptation actions; (ii) resilience as a mission-level capability that continuously preserves operational effectiveness despite interruptions. Rather than treating monitoring, recovery, and re-orchestration as isolated activities, we unify them as interconnected processes within a resilience-oriented orchestration lifecycle [1, 16]. This paper is guided by the following research questions that are relevant to the current autonomous MAS research:

- **RQ1** - To what extent does the proposed framework preserve mission continuity under increasing failure rates and failure criticalities?
- **RQ2** - How does the proposed framework compare with baseline recovery strategies in terms of recovery success?
- **RQ3** - How effectively does the proposed framework perform runtime feasibility assessment in preventing invalid or unsafe standby promotions?



This work is licensed under a Creative Commons Attribution 4.0 International License. RSSR '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2992-8/2026/10
<https://doi.org/10.1145/3843780.3844537>

To address these research questions, we propose **AHToViK**, an Asynchronous Heterogeneous Task Orchestration via a modified VIKOR framework. The main focus of this work is guided towards automated resilience mechanisms for safety-critical autonomous MAS, with runtime feasibility assessment preventing unsafe recovery actions. This is achieved by unifying runtime monitoring, interruption detection, recovery planning, recovery re-assessment, and re-orchestration. The main contributions of this paper are as follows.

- An event-driven lifecycle for mission continuity in autonomous MAS.
- A standby promotion strategy based on modified-VIKOR ranking and runtime re-assessment.
- A runtime feasibility assessment that filters unsafe promotions before re-orchestration.

The novelty of **AHToViK** is not in introducing new runtime monitoring, recovery planning, or recovery re-assessment mechanisms individually, as these capabilities are well established in resilient and self-adaptive systems. Instead, **AHToViK** redefines resilience as an orchestration property that emerges when monitoring, recovery, re-assessment, and re-orchestration are unified into a single lifecycle. Conventional self-adaptive architectures, such as MAPE-K [10], are adaptation architectures whose primary objective is system adaptation and restoring a valid operational configuration when deviations are detected. In contrast, **AHToViK** is an orchestration architecture whose primary objective is mission continuity and the preservation of the operational state through resilience-aware execution. To achieve this, stages from planning to re-orchestration are designed as a unified mission continuity lifecycle, in which each stage directly propagates its outcome to the next. Consequently, mission continuity is no longer treated as the result of independent monitoring and recovery mechanisms but as an architectural property that emerges from a unified resilience-oriented orchestration process. This enables recovery decisions to be part of the orchestration process itself, allowing mission execution to continue via verified runtime reconfiguration rather than suspending execution until a separate adaptation cycle completes.

The remainder of this paper is organized as follows. Section 2 reviews existing research on runtime monitoring, self-healing recovery, and resilience in multi-agent systems. Section 3 presents the proposed **AHToViK** framework and resilience lifecycle. Section 4 describes the experimental setup and evaluation methodology. It also discusses an analysis of resilience performance. Finally, Section 5 concludes the paper and outlines future research directions.

2 Related Work

In this section, we approach the literature from the perspective of mission continuity and resilience, focusing on three key research areas based on our research questions introduced in Section 1: (i) runtime monitoring and disruption detection, (ii) recovery and self-healing mechanisms, and (iii) recovery re-assessment for resilient re-orchestration.

Runtime Monitoring and Disruption Detection (RQ1). The primary mechanism in autonomous systems for monitoring mission execution and detecting variations in expected behavior is runtime

monitoring. The existing literature has proposed various monitoring approaches for autonomous MAS, including health monitoring, fault diagnosis, runtime verification, telemetry analysis, and adaptive feedback control [1]. These mechanisms perform a continuous assessment of agent status, communication quality, resource availability, environmental conditions, and task progress to identify anomalies during mission execution. However, existing monitoring approaches often focus on failure detection rather than mission continuity preservation. Monitoring outputs are often treated as diagnostic information that is interpreted by separate adaptation or recovery mechanisms. Thus, even though interruptions can be identified, mapping these monitoring outcomes into mission-level recovery decisions is still weakly integrated in most cases. This generates a gap between interrupt detection and mission continuation in dynamic environments where recovery decisions must be initiated rapidly to prevent failure propagation.

Recovery and Self-Healing Mechanisms (RQ2). In the literature, there has been extensive research into recovery and self-healing mechanisms aimed at improving the fault tolerance and robustness of autonomous MAS. Existing approaches include task reallocation, coalition reformation, adaptive re-planning, role reassignment, redundancy-based recovery, and fault-tolerant coordination strategies. These methods aim to restore operational functionality after interruptions through task re-allocation, replacement of failed agents, or adaptation of execution plans to evolving mission conditions [11, 17]. More recently, self-adaptive systems research has incorporated MAPE-K-based adaptation loops and autonomous recovery policies to improve resilience during execution. Although these approaches improve robustness, recovery is typically treated as a localized response to failures rather than a mission continuity objective. Many recovery mechanisms focus on restoring failed components or generating alternative allocations without explicitly evaluating how recovery decisions affect the overall progress of the mission [8, 13]. As a result, maintaining mission continuity depends on the successful interaction of multiple independent recovery mechanisms, not on a unified resilience process.

Recovery Re-assessment and Resilient Re-Orchestration (RQ3). As MAS have emerged as responsible for critical mission execution, one of the major concerns that has arisen is to ensure the feasibility and operational effectiveness of recovery actions [11, 12]. Consequently, runtime verification, assurance monitoring, safety validation, and adaptive verification techniques have emerged as complementary mechanisms to evaluate adaptation and recovery decisions. Existing approaches often verify constraint satisfaction, safety properties, resource availability, or mission requirements before executing adaptation actions [1, 2, 16]. Verification is frequently implemented as a separate activity that operates independently of the orchestration and recovery processes. Recovery decisions are often generated and then validated later through dedicated verification mechanisms. In turn, verification ensures the correctness of the framework. However, a key limitation remains, as verification rarely operates as an active component of the resilience process itself. This separation limits the ability of autonomous systems to continuously govern recovery decisions according to evolving mission conditions and mission continuity requirements [6].

Discussion: Coordination in MAS and the orchestration of self-adaptive systems have each been studied for over three decades, and

substantial individual advances in runtime monitoring, recovery mechanisms, self-adaptive coordination, and runtime verification have been made [15]. However, these capabilities are commonly evaluated as independent runtime functions. Therefore, the relationships among interrupt detection, recovery activation, recovery re-assessment, and mission continuation are weakly integrated within existing orchestration frameworks. Table 1 shows a comparison of AHToViK among the frameworks discussed in this section. Existing studies address individual aspects of RQ1 (runtime monitoring and failure detection), RQ2 (recovery planning), or RQ3 (recovery re-assessment), but none integrates all three research dimensions into a unified resilience-oriented lifecycle.

3 Methodology

The primary objective of this section is to explain how AHToViK preserves mission continuity under operational interruptions and how recovery actions are generated, validated, and integrated into the mission execution process. Understanding these resilience mechanisms requires a basic understanding of the underlying AHToViK decision logic. Recovery decisions are derived directly from the initial task orchestration process. Therefore, an overview of the AHToViK mission orchestration framework to establish the decision-making principles relevant to the proposed resilience-oriented lifecycle is briefly explained in section 3.1.

3.1 AHToViK Mission Orchestration Framework

AHToViK is a decentralized coordination framework for dynamic task assignment in heterogeneous MAS. Unlike traditional static allocation approaches, it continuously adapts task assignments to changing mission conditions via an asynchronous, event-driven backbone, as illustrated in Figure 1. We propose AHToViK as an orchestration rather than choreography framework because agents do not merely exchange peer-to-peer messages here. They compute and apply the same ranking logic and lifecycle, with the AEB's occupancy events ensuring conflict-free promotion as a centralized framework would do. In AHToViK, execution is distributed across agents, but the decision model and lifecycle are unified and prescribed, unlike independent negotiations in choreography. The framework operates as a four-stage pipeline. First, a task prioritization engine receives mission requirements and tasks to be performed by the heterogeneous fleet. Then, it selects relevant decision criteria from a broader set through context-aware criteria selection. These criteria are compared pairwise and weighted using a scheme based on the Analytic Hierarchy Process (AHP), producing a verified priority vector (W) that weights each decision criterion based on the relevance of the specific task. This stage performs a consistency ratio check ($CR \leq 0.1$) that acts as a logical circuit breaker, rejecting the weighting if the underlying judgments are inconsistent. Second, using the same selected criteria, the geometric mapper populates the decision matrix from real-time robot telemetry to compute each available agent's ideal distances, its group utility (D_i^+), and individual regret (D_i^-). Third, the Team Evaluator module calculates the compromise index (Q) to rank agents based on suitability for the tasks (ordered based on priority). It categorizes the agents into primary and standby niches based on their compromise index. This stage provides a final task assignment list. Finally,

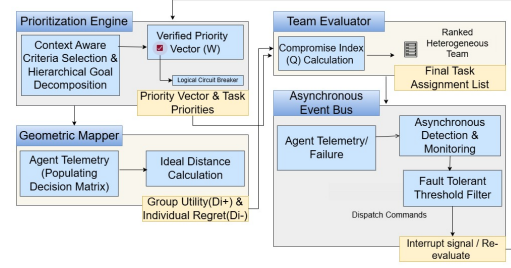


Figure 1: AHToViK - High-level architecture

the Asynchronous Event Bus (AEB) module achieves coordination and runtime adaptation. The formalization of the modified VIKOR index used for calculating the fitness score for each agent is given below:

$$Q = v \left(\frac{D_i^+ - D_{i\min}^+}{\Delta D_i^+} \right) + (1 - v) \left(\frac{D_{i\max}^- - D_i^-}{\Delta D_i^-} \right) \quad (1)$$

Where v denotes the weight of the majority strategy in the VIKOR index calculation formula, or here, the maximum group utility. $D_{i\min}^+$ denotes the smallest distance from the ideal best found across the entire fleet. $D_{i\max}^-$ denotes the highest safety margin, the one furthest from the ideal worst, to mitigate operational failure. ΔD_i^+ and ΔD_i^- represent the total spread of closeness to perfection within the fleet and the total spread of safety margins within the fleet, respectively. These values are calculated as the difference between the corresponding max and min values. The formalization and the sensitivity analysis for the modified Q index are available in the replication package¹. AHToViK follows a distributed logic model in which decision-making is performed locally by individual agents rather than a centralized coordinator. The calculated Q rankings are replicated across participating agents, providing a shared view of team suitability while allowing the AEB to remain stateless. During execution, the AEB continuously disseminates operational events and monitors niche stability. When a significant change in agent suitability is detected, the next-best-ranked agent is autonomously promoted to preserve mission continuity without requiring global task reallocation. The decision logic for task prioritization and team formation is a separate contribution of AHToViK, outside the scope of this paper, and available in the replication package.

3.2 AEB as the Mission Continuity Backbone

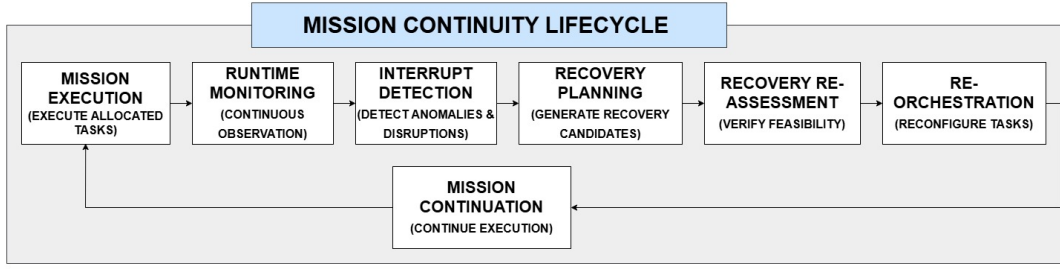
The Asynchronous Event Bus (AEB) is implemented as a stateless event-driven coordination backbone that enables decentralized orchestration among autonomous agents. Instead of maintaining global system state, it disseminates transient operational events describing changes in agent health, resource availability, communication status, task progress, and fitness scores (Q -values). Consequently, every agent independently interprets the current operational context using locally available information, allowing autonomous role adaptation without centralized coordination while preserving mission continuity under dynamic conditions.

Within AHToViK, each agent subscribes only to the task-specific topics associated with its operational responsibilities and publishes

¹<https://doi.org/10.5281/zenodo.21028466>

Table 1: Comparison of representative related work with respect to the research objectives addressed in this paper.

Representative approach	RM	FD	RP	RR	RO	MC	UL
Adaptive Task Planning with LLMs [11]	Partial	Partial	Yes	No	Partial	No	No
Distributed Resilient Consensus [17]	Partial	Yes	Yes	No	No	No	No
Task Coordination and Recovery [5]	Yes	Yes	Yes	No	No	No	No
Runtime Composition Framework [1]	Yes	Partial	Partial	Yes	No	No	No
Probabilistic Verification [16]	Partial	No	No	Yes	No	No	No
Priority Based Fault Tolerant Scheduling [13]	No	Yes	Yes	No	No	No	No
Collaborative Robot Orchestration [12]	Partial	Partial	Yes	Partial	Yes	No	Partial
Robust MAS Verification [2]	No	No	No	Yes	No	No	No
Learning Safe Control and Verification [6]	Partial	No	No	Yes	No	No	No
Consensus-Based Decentralized Task Allocation [4]	No	Partial	Yes	No	Partial	No	No
AHToViK	Yes	Yes	Yes	Yes	Yes	Yes	Yes

**Figure 2: AHToViK Mission continuity lifecycle**

updated operational information whenever significant changes occur. Upon receiving an event, agents locally evaluate the decision criteria, compute their compromise indices (Q values), and publish the resulting fitness values. To preserve decentralization while satisfying the global normalization requirement of the AHToViK decision model, a lightweight task-scoped aggregator computes only the shared extrema (D_i^+ and D_i^-) required for consistent local Q_i calculation. The aggregator is instantiated independently for each task and operates as a lightweight transient synchronization service rather than a centralized controller. Its sole responsibility is to compute the shared extrema required for local Q -value normalization, after which every participating agent independently performs recovery re-assessment and determines its own promotion eligibility. Since the aggregator maintains no persistent orchestration state, its temporary unavailability only postpones the computation of updated extrema without affecting the current mission configuration; a new aggregator instance can be recreated from the latest published agent states.

Agents are organized into *active* and *standby* niches according to their compromise index (Q), where the highest-ranked agent occupies the active operational role while the remaining candidates remain in standby. When an active agent violates an operational threshold or becomes unavailable, the corresponding niche is declared vacant, initiating the mission continuity lifecycle. To prevent conflicting promotions, each operational niche is associated with a unique occupancy event disseminated through the AEB, allowing only the first valid promotion event to reserve the vacant niche

while concurrent promotion attempts are ignored. Throughout this paper, an *unsafe promotion* refers to promoting a standby agent without completing the implicit recovery re-assessment, whereas an *invalid promotion* denotes promoting an agent that is ultimately unsuitable for the required operational role. Finally, the communication losses are detected through peer-observed expiration of periodic operational updates (heartbeat/lease mechanism), after which subscribed agents locally classify the affected agent as unavailable and publish the corresponding vacancy event to the AEB.

3.3 Fault-Tolerant Re-Allocation and Recovery

Mission continuity within autonomous MAS requires the ability to recover from unexpected operational disruptions without introducing centralized recovery procedures. Whenever an active agent becomes unavailable due to communication loss, capability degradation, resource depletion, or any other operational failure, the AEB immediately disseminates a failure event indicating that the corresponding operational niche has become vacant. Instead of waiting for centralized synchronization or timeout mechanisms, the remaining agents autonomously interpret this event and initiate local recovery decisions. To better understand the mission continuity lifecycle, we walk through a flexible production line case study as follows.

Running Case Study-Flexible Production Line. To illustrate the AHToViK mission continuity lifecycle shown in Figure 2, consider a flexible Industry 5.0 production line comprising a heterogeneous team of autonomous robots responsible for fulfilling a

manufacturing order-101. The production process involves a mobile transport robot delivering raw components to a robotic assembly station, followed by a welding robot performing structural welding and an inspection robot verifying weld quality before the component proceeds to the next manufacturing stage. Before execution, AHToVik performs task prioritization and team formation, assigning both active and standby robots for each operational role. The execution of a welding task in which an active welding robot experiences an unexpected overheating fault during production is explained for the convenient understanding of AHToVik. It demonstrates how AHToVik preserves mission continuity through runtime monitoring, interruption detection, recovery planning, recovery re-assessment, and resilience-oriented re-orchestration, ultimately allowing a standby robot to assume the operational role and continue production with minimal disruption.

Runtime Monitoring with AEB. Following the initial task prioritization and team formation, AHToVik assigns the welding robot of highest rank (based on Q -value), say W_1 , to the active operational niche for a manufacturing order-101, while the next welding robot of highest rank, W_2 , remains in the waiting niche. As depicted in Figure 3, the mission begins with the Runtime Monitoring stage, where the AEB continuously receives and disseminates operational events published by W_1 . These events collectively illustrate the key aspects of the robot's execution state: *Agent Health*, including actuator status, battery level, and operating temperature; *Resource*, including tool availability, welding material, and energy consumption; *Communication*, including connectivity and communication quality with the remaining team members; and *Task Progress*, including completed weld segments, current welding operation, and remaining work required to complete the assigned task. As long as all monitored parameters remain within their expected operational limits, the welding task proceeds uninterrupted.

Suppose during the welding task, the operating temperature of W_1 gradually exceeds its allowable threshold due to prolonged execution. This abnormal telemetry generates an operational event indicating that one or more monitored parameters have deviated from their acceptable limits. The AEB immediately propagates this interruption event to all agents participating in the welding task. This leads the mission continuity lifecycle to transition from runtime monitoring to the interrupt detection stage, where AHToVik analyzes the propagated interruption to determine whether it represents a transient operational anomaly or a genuine failure requiring recovery and resilience-oriented re-orchestration.

Interruption Detection and Mission Impact Assessment. Following the overheating event propagated by the AEB during runtime monitoring, the interruption detection stage determines whether the observed deviation represents a temporary operational anomaly or a recoverable failure capable of affecting mission continuity. From Figure 3, the process begins with *Anomaly Detection*, where the continuously increasing operating temperature of the active welding robot W_1 is identified as a deviation from its normal operating behavior. *Threshold Check* component then verifies whether the reported temperature exceeds the predefined safe operating limit specified by W_1 's operational specification. Since these operational thresholds are application-specific rather than determined by AHToVik, they represent the safety constraints under which the robot can safely execute the assigned task. Once a threshold

breach is detected, the updated operational parameters are propagated through the AEB, allowing the affected robot to recompute its fitness score (Q) using the AHToVik decision logic. The newly computed ranking is compared against the previously established operational ranking using the stability threshold obtained through the offline sensitivity analysis. If the deviation exceeds the accepted tolerance (10%), indicating that the robot cannot reliably fulfill its assigned operational role, the interruption is classified as a *niche breach*. Otherwise, the event is regarded as a transient operational fluctuation and execution returns to the runtime monitoring stage without initiating recovery.

To distinguish persistent failures from temporary disturbances, the *Trend Analysis* component evaluates the temporal evolution of the monitored parameters instead of reacting to a single abnormal measurement. In this case, if the operating temperature of W_1 continues to rise over successive monitoring intervals, the interruption is confirmed as a persistent degradation rather than a momentary thermal fluctuation. Once confirmed, the *Mission Impact Assessment* component evaluates the operational consequences of the interruption by considering four factors: (i) *task criticality*, representing the importance of the interrupted welding task to the manufacturing order; (ii) *dependency impact*, representing the effect of the interruption on downstream assembly and inspection operations; (iii) *resource availability*, representing the availability of suitable standby welding robots; and (iv) *expected mission delay*, representing the production delay introduced if recovery is performed. Since the interrupted welding task directly influences subsequent manufacturing stages, the interruption is assigned a high recovery priority. After mission impact assessment, the interruption is classified as a recoverable operational failure, and the AEB disseminates the corresponding failure event to all team members in the affected task. Instead of initiating a complete reallocation of manufacturing order-101, the mission continuity lifecycle proceeds to the *Recovery Decision and Candidate Generation* stage, where the orchestration context established during the initial task prioritization and team formation is exploited to identify the most suitable standby welding robot for uninterrupted mission continuation.

Recovery Planning and Candidate Generation. Once the interruption affecting the active welding robot W_1 is classified as a recoverable operational failure, the mission continuity lifecycle transitions to the recovery planning stage. Rather than initiating the reallocation of order-101 again, AHToVik exploits the orchestration context established during the initial task prioritization and team formation. Using the updated interruption event disseminated through the AEB, the framework identifies the standby welding robots already associated with the affected task and retrieves their most recent operational information, including their capability status, resource availability, communication status, and fitness score (Q). Based on this information, the framework generates feasible recovery candidates capable of assuming the interrupted welding operation without requiring centralized re-planning. For each candidate, AHToVik estimates the operational impact of the proposed recovery by considering the expected interruption to the manufacturing workflow, resource utilization, recovery delay, and the effect on downstream assembly and inspection operations. Candidate recovery configurations are then prioritized according to their estimated mission disruption, where the configuration introducing

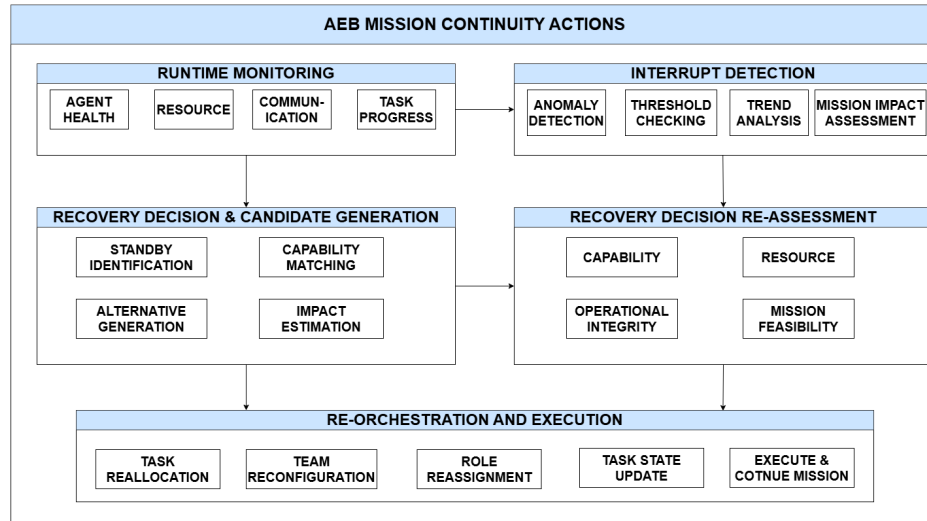


Figure 3: AEToVik Mission continuity activities performed by AEB

the least operational impact is selected as the preferred recovery candidate. The selected standby welding robot is subsequently forwarded to the recovery re-assessment stage, where its operational readiness is verified before promotion to the active operational role.

Recovery Re-assessment. Once the recovery planning stage selects a preferred standby, say W_2 , the agent does not immediately assume the vacant operational role. Instead, it performs a recovery re-assessment by re-evaluating its current fitness score using the latest operational information disseminated through the AEB. Here, a promoted standby robot is verified along four dimensions before it takes over a failed task. These are the robot's capabilities to operate, the availability of the resources, the operational integrity measured by environmental factors within safe operating thresholds, and the robot's feasibility to execute the mission. This re-assessment re-executes the implicit decision-assessment activities already embedded within the AEToVik decision logic. As a result, changes in resource availability, communication quality, environmental conditions, task requirements, or robot capabilities are automatically reflected in the recomputed fitness score (Q), allowing the framework to determine whether W_2 is still the most suitable replacement for the interrupted welding task. After the re-assessment, all other standby robots subscribed to the welding task publish their ideal best and worst values to the AEB, where the task-scoped aggregator computes the shared extrema required for the fitness score calculation and identifies the best-fit recovery candidate. If W_2 retains the lowest compromise index after re-assessment, it is approved for promotion to the active operational niche; otherwise, the next most suitable standby robot is considered. By reusing the implicit decision-assessment activities instead of introducing a separate verification module, AEToVik ensures that recovery decisions remain logically consistent, operationally feasible, and aligned with the evolving mission state before progressing to the Re-orchestration stage. The software engineering formalization of these implicit verification activities is provided in the replication package².

Resilience-Oriented Re-orchestration and Mission Continuity Preservation.

After successful recovery re-assessment, the selected standby welding robot, W_2 , is promoted to the active operational niche and the mission continuity lifecycle proceeds to the re-orchestration stage. As depicted in Figure 3, re-orchestration transforms the assessed recovery decision into executable mission actions by updating task assignments, reconfiguring the welding team, reassigning operational roles, and synchronizing the revised orchestration context among the affected agents. The AEB disseminates these updated operational events, enabling every subscribed agent to autonomously transition to the new mission configuration without requiring centralized coordination or complete mission re-planning. For manufacturing order-101, the interrupted welding task is reassigned from the failed robot W_1 to the standby robot W_2 . Since the orchestration context established during mission execution, including current task assignment, team composition, operational roles, capability status, and updated fitness rankings, is continuously maintained through the AEB, W_2 can immediately assume responsibility for the interrupted welding operation without repeating the initial task allocation process. The production workflow therefore continues with minimal disruption, allowing the downstream assembly and inspection operations to proceed according to the updated team configuration. Once re-orchestration is completed, the promoted robot resumes normal execution and re-enters the runtime monitoring stage, completing one iteration of the proposed mission continuity lifecycle shown in Figure 2. Therefore, runtime monitoring, interruption detection, recovery planning, recovery re-assessment, and resilience-oriented re-orchestration operate as a unified event-driven orchestration process, where each stage directly propagates its outcome to the next to preserve mission continuity despite dynamic operational disruptions.

4 Simulation and Validation

In this section, we describe a simulation environment created in Google Colab and Gazebo/ROS2 Humble using Python for a search

²<https://doi.org/10.5281/zenodo.21028466>

and rescue (SAR) scenario. For simulation, a critical mission scenario is considered to better explore the algorithm's decision-making logic under high-stakes conditions, while the welding robot overheating case in section 3.3 is presented earlier as an illustrative example to help readers intuitively understand the recovery process. Consider the scenario where, after a major earthquake, a heterogeneous multi-robot system consisting of Unmanned Aerial Vehicles (UAVs), Unmanned Ground Vehicles (UGVs), and tracked creeper robots is deployed to assess structural damage and locate trapped survivors. Multiple mission objectives are generated simultaneously, including victim detection, debris inspection, medical supply delivery, and structural monitoring. AHToVIK first determines the relative priority of these tasks according to the operational objectives and forms task-specific teams. The implementation and validation results are available in the replication package³.

Ablation Studies. To better understand the contribution of each architectural component within AHToVIK, we conducted a mechanism-aware ablation study over 500 simulations, in which individual modules are systematically removed while the remaining framework is left unchanged. Each removed component is paired with the metric that most directly reflects its intended functionality, while mission continuity is additionally reported for all variants as an overall system-level resilience indicator. Mission continuity is computed as follows.

$$MC (\%) = \frac{N_{\text{completed}}}{N_{\text{total}}} \times 100 \quad (2)$$

where $N_{\text{completed}}$ is the number of missions completed without an unrecovered interruption and N_{total} is the total number of simulated missions.

Removing runtime monitoring does not eliminate failure awareness outright or leave the system flying blind. It replaces the AEB's continuous, event-driven telemetry with periodic inspection every five mission steps at a 60% per-check detection probability, so failures are both delayed and, in a fraction of cases, never detected within the entire fleet. This compounded latency and recall loss leads to 3.19 undetected failures per mission. It also reduces mission continuity to 52.84%. Eliminating recovery re-assessment allows 2.58 unsafe promotions per mission, reducing mission continuity to 77.23%. This confirms that re-assessment is necessary to filter unsuitable standby candidates before promotion. Without the task-scoped aggregator, decision execution time rises from 8.17 ms to 35.91 ms due to added communication and synchronization overhead. This results in marginal continuity impact (77.75%) as the aggregator's role is computational, not continuity-critical. Finally, removing the recovery mechanism entirely establishes a lower-bound resilience baseline of 39.12%, confirming that these mechanisms act collectively, rather than individually, to sustain robust mission execution.

Validation against Baselines. To evaluate the overall effectiveness of AHToVIK as a unified resilience-oriented orchestration lifecycle, it is compared against five recovery strategies such as CBBA-based recovery (decentralized consensus) [4], threshold rule recovery (rule-based), MAPE-K recovery (self-adaptive control loop), centralized re-planning, and risk-aware greedy recovery. We

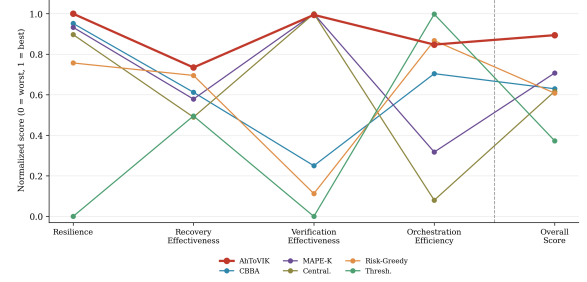


Figure 4: AHToVIK comparison results with baselines

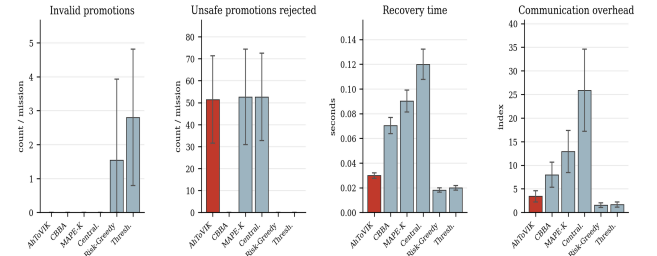


Figure 5: Selected underlying raw metrics used for calculation of evaluation factors

configure the baselines for validation with threshold rule recovery promoting next-ranked standby once a fixed 10% deviation threshold is breached; risk-aware greedy recovery selecting a standby with the lowest immediate risk score from current telemetry; centralized re-planning invoking a global optimizer over the full assignment matrix per interruption; and MAPE-K recovery executing the four MAPE-K phases sequentially using the same telemetry inputs as AHToVIK, without feasibility re-assessment. Full parameters and implementation code for all baselines are available in the replication package⁵.

All approaches are evaluated on the same heterogeneous fleet of 30, 50, 100, 200, and 300 robots assigned to 20 concurrent tasks. The robots are injected with a failure probability of 0.3 per robot over the mission horizon. Each recovery strategy independently selects and promotes standby replacements according to its own decision logic. All six approaches are simulated over 500 independent runs under this identical configuration, and each run is logged along sixteen metrics spanning mission outcome (such as mission continuity, completion rate, downtime, cascading failures), recovery behavior (such as recovery success rate, recovery time, mission delay, re-allocations), promotion safety (such as invalid promotions, unsafe promotions rejected, verification precision and recall), and coordination cost (such as message count, event latency, runtime, communication overhead). The sixteen metrics are grouped into four validation factors for the ease of representation, and the comparison results are shown in Figure 4.

The overall score for each approach is calculated as the unweighted mean of four evaluation factors such as Resilience, Recovery Effectiveness, Verification Effectiveness, and Orchestration

³<https://doi.org/10.5281/zenodo.21028466>

⁴Mission continuity itself is the evaluation criterion for the Recovery ablation.

⁵<https://doi.org/10.5281/zenodo.21028466>

Table 2: Mechanism-aware ablation study of AHToVik (mean values over 500 runs).

Removed Component	Evaluation Criterion	Full	Ablated	Mission Continuity (%)
Runtime Monitoring	Undetected failures (count/mission)	0	3.19	52.84
Recovery Re-assessment	Unsafe promotions (count/mission)	0	2.58	77.23
Task-Scoped Aggregator	Decision execution time (ms)	8.17	35.91	77.75
Recovery	Mission continuity (%) ⁴	77.77	39.12	39.12

Efficiency. Each of these factors is calculated by min-max normalization of the evaluation metrics across six approaches, some of which are shown in Figure 5. For example, resilience is the mean of mission continuity, completion rate, downtime, and cascading failures. AHToVik attains the highest overall score (0.894), ahead of MAPE-K (0.707), CBBA (0.630), centralized re-planning (0.617), and risk-aware greedy (0.608), and threshold rule recovery with the lowest (0.373). The overall score is the unweighted mean of the four evaluation factor scores. The detailed computation results for sixteen underlying metrics and four evaluation factors are available in the replication package due to space limitations.

Threats to Validity. The validity of our results is subject to a set of limitations. Internal validity may be affected by the configuration of the simulated agents, failure-injection mechanisms, operational thresholds, and task parameters, which may influence both recovery activation and mission-continuity measurements. To reduce this threat, all strategies were evaluated under the same conditions across 500 independent runs. Construct validity concerns the metrics used to represent resilience. Mission continuity, recovery success, and invalid promotions capture relevant aspects of resilient execution but do not fully account for detection latency, communication overhead, energy consumption, or the severity of partially completed missions. External validity is limited by the use of a simulated ROS 2/Gazebo SAR scenario and a specific heterogeneous robot configuration. Thus, the results may not directly generalize to different application domains, physical deployments, or adversarial disruptions. Future work will evaluate AHToVik in real-world settings and under additional failures and attacks, including stale or manipulated telemetry, communication jamming, and concurrent agent failures.

5 Conclusion

In this paper, we present a resilience-oriented orchestration framework for preserving mission continuity in autonomous multi-agent systems operating under dynamic conditions. The framework integrates runtime monitoring, interruption detection, recovery planning, recovery re-assessment, and re-orchestration within a unified mission continuity lifecycle to support resilient mission execution. Experimental results indicate that the proposed approach can effectively maintain mission continuity and reduce unsafe recovery decisions across the evaluated scenarios. Future work will investigate large-scale deployments, adaptive recovery strategies, and task state preservation in more complex mission environments.

References

- [1] Muhammad Ashfaq, Ahmed R. Sadiq, Teerath Das, Muhammad Waseem, Niko Mäkitalo, and Tommi Mikkonen. 2026. Runtime composition in dynamic system

- of systems: A systematic review of challenges, solutions, tools, and evaluation methods. *Journal of Systems and Software* 232 (2026), 112661. doi:10.1016/j.jss.2025.112661
- [2] Raphaël Berthon, Joost-Pieter Katoen, Munyque Mittelmann, and Aniello Murano. 2026. Verification of Robust Multi-Agent Systems. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 3758–3766.
- [3] Rafael Cardoso, Brian Logan, Felipe Meneguzzi, Nir Oren, and Bruno Yun. 2022. Resilience, reliability, and coordination in autonomous multi-agent systems. *AI Communications* 35 (09 2022), 1–18. doi:10.3233/AIC-220136
- [4] Han-Lim Choi, Luc Brunet, and Jonathan P. How. 2009. Consensus-Based Decentralized Auctions for Robust Task Allocation. *IEEE Transactions on Robotics* 25, 4 (2009), 912–926. doi:10.1109/RO.2009.2022423
- [5] Federica Filippini, Marin Lujak, and Michele Ciavotta. 2026. Distributed replica allocation and load balancing for Edge–Cloud FaaS: A cooperative multi-agent orchestration approach. *Journal of Systems Architecture* 176 (2026), 103787. doi:10.1016/j.sysarc.2026.103787
- [6] Kunal Garg, Songyuan Zhang, Oswin So, Charles Dawson, and Chuchu Fan. 2024. Learning safe control for multi-robot systems: Methods, verification, and open challenges. *Annual Reviews in Control* 57 (2024), 100948. doi:10.1016/j.arcontrol.2024.100948
- [7] Debanjan Ghosh, Raj Sharman, H. Raghav Rao, and Shambhu Upadhyaya. 2007. Self-healing systems — survey and synthesis. *Decision Support Systems* 42, 4 (2007), 2164–2185. Decision Support Systems in Emerging Economies. doi:10.1016/j.dss.2006.06.011
- [8] Andriansyah Hamid, Yasser Almoghathawi, Anas Alghazi, and Haitham Saleh. 2025. Enhancing resilience in UAV swarms: A literature review. *Journal of Safety Science and Resilience* (2025), 100268. doi:10.1016/j.jnlsr.2025.100268
- [9] Xin Jin, Tengjiao Guo, Dazhi Zhan, and Qi Cao. 2026. From simulation to learning: A comprehensive review of multi-agent systems in emergency management. *Neurocomputing* 692 (2026), 133841. doi:10.1016/j.neucom.2026.133841
- [10] J.O. Kephart and D.M. Chess. 2003. The vision of autonomic computing. *Computer* 36, 1 (2003), 41–50. doi:10.1109/MC.2003.1160055
- [11] Jonghan Lim, Jiabao Zhao, Ezekiel Hernandez, and Ilya Kovalenko. 2026. Adaptive task planning and coordination in multi-agent manufacturing systems using large language models. *Robotics and Computer-Integrated Manufacturing* 100 (2026), 103245. doi:10.1016/j.rcim.2026.103245
- [12] Ziheng Liu, Wenjun Xu, Bitao Yao, Lei Qi, and Mengyuan Ba. 2026. A Hierarchical Framework of Central-Distributed LLM Negotiation and Specialized Model Orchestration for Multi-Robot Collaborative Assembly. *Procedia CIRP* 142 (2026), 728–733. 36th CIRP Design Conference (CIRP Design 2026). doi:10.1016/j.procir.2026.05.337
- [13] Sheikh Umar Mushtaq, Sophiya Sheikh, and Sheikh Mohammad Idrees. 2025. Enhanced priority based task scheduling with integrated fault tolerance in distributed systems. *International Journal of Cognitive Computing in Engineering* 6 (2025), 152–169. doi:10.1016/j.ijcce.2024.12.006
- [14] Sebastian Rodriguez, Akhila Bairy, Matteo Baldoni, Patrick Benjamin, Constantin Blessing, Nicolas Brandstetter, Amit K. Chopra, Thomas Clemen, Louise A. Dennis, Ahmad Esmaili, Lu Feng, Angelo Ferrando, Zahra Ghorrati, Victor Guillet, Önder Gürçan, Soham Hans, James Herber, Viviana Mascardi, Marcel Mauri, Jörg P. Müller, John Thangarajah, Rafal Tyl, and Yi Yang. 2026. Engineering the Next Generation of Multi-agent Systems: A Community Roadmap from EMAS 2025. In *Engineering Multi-Agent Systems*, Sebastian Rodriguez, Lu Feng, and Jörg P. Müller (Eds.). Springer Nature Switzerland, Cham, 238–258.
- [15] Mazeiar Salehie and Ladan Tahvildari. 2009. Self-adaptive software: Landscape and research challenges. *ACM Trans. Auton. Adapt. Syst.* 4, 2, Article 14 (May 2009), 42 pages. doi:10.1145/1516533.1516538
- [16] Xia Wang, Jun Liu, Chris D. Nugent, Shaobing Xu, and Guanfeng Wu. 2026. Team collaboration-oriented multi-agent pathfinding and probabilistic verification. *Information Fusion* 130 (2026), 104125. doi:10.1016/j.inffus.2026.104125
- [17] Yuhang Yang, Daduan Zhao, Jin You, Yangquan Chen, and Yan Li. 2026. Distributed resilient consensus for heterogeneous fractional-order multi-agent systems against hybrid attacks. *ISA Transactions* (2026). doi:10.1016/j.isatra.2026.06.021

Received 2026-07-19; accepted 2026-08-25